

PLATFORM ENGINEERING

De OpenClaw a EnterpriseClaw

Evolución del Platform Engineering hacia la Orquestación de Agentes

Juan David Argüello

InnerSource Program Lead · Credicorp





Juan David Argüello

InnerSource Program Lead · Credicorp

- ▶ **MSc in Computer Science** — University of Bath, UK
- ▶ **Postgrado en AI & ML** — The University of Texas at Austin
- ▶ **Kubestronaut** — en camino al Golden Kubestronaut

KUBESTRONAUT

INNERSOURCE

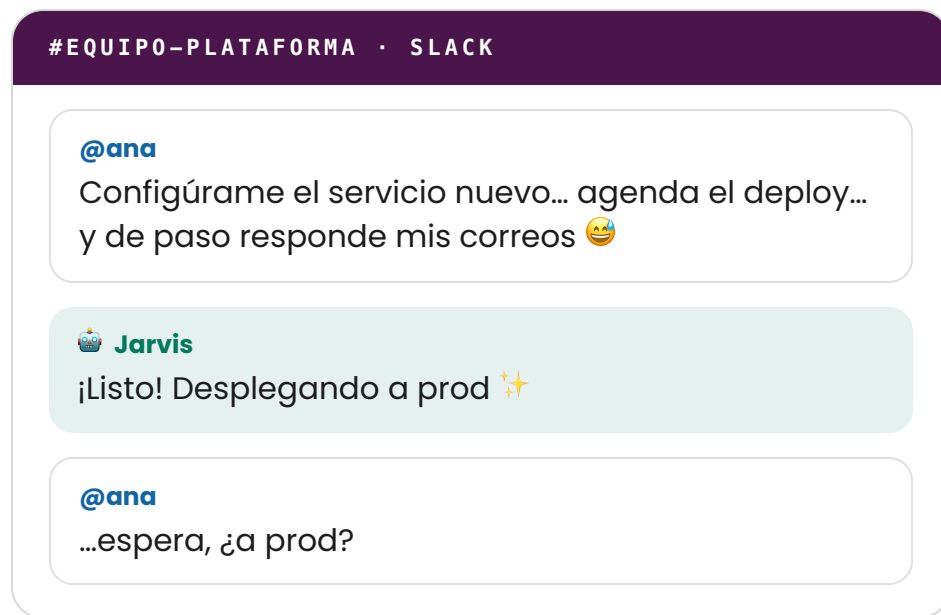
PLATFORM ENG

/in/jdarp

Acto 1: 2026, el año de los agentes



EL SUEÑO



OpenClaw — el framework open source de agentes personales que se volvió viral: de chats que *responden* a agentes que *actúan* a nombre del usuario. Tanto, que ya se le considera **«el Linux del mundo agéntico»**.

*Todos queremos nuestro propio Jarvis.
Y ahora todos lo quieren... en la oficina.*

Ahora...



¿Por qué **OpenClaw** es disruptivo?

¿Cómo funcionaría en un **entorno empresarial**?



Potenciando el SDLC

EL PUENTE – DEL JARVIS PERSONAL AL CICLO DE DESARROLLO

¿Y si «ordenar en lenguaje natural → el agente actúa» tocara cada etapa de cómo construimos software? Agentes que **redactan issues** desde la historia de la organización · **escriben y validan código** en CI · **prueban funcionalmente** en dev y QA · **proponen la infraestructura** · hacen **triage en producción**.



La disrupción no es el chat.

Cada espera humana en el SDLC es reloj corriendo. Y cuando producción falla, el fallo es un evento más: se observa, se captura y se remedia por el mismo ciclo – comprimir los relojes también comprime el MTTR.

¡Genial!



¿Cómo hago para **instalarlo en la empresa?**

Aunque... ¿por qué **no he escuchado de compañías**
que lo estén utilizando?



El gap: de tu laptop a la empresa

En tu máquina personal, un agente comprometido es *tu* problema. En una empresa: **miles de empleados · datos regulados · acceso a producción · auditores.**

ESCENARIO

- "Solo basta con que **uno** de los miles de empleados abra un email indebido en el momento incorrecto..." — un agente con acceso amplio, secuestrado por instrucciones de un atacante.



MIEDO 01

Prompt injection

Una instrucción hostil escondida en el contenido secuestra el razonamiento del agente.



MIEDO 02

Suplantación de usuario

¿Quién pidió esto — el humano, o un bot con permisos de superusuario?



MIEDO 03

Fuga de secretos

El agente ve credenciales y datos que nunca debió poder ver.

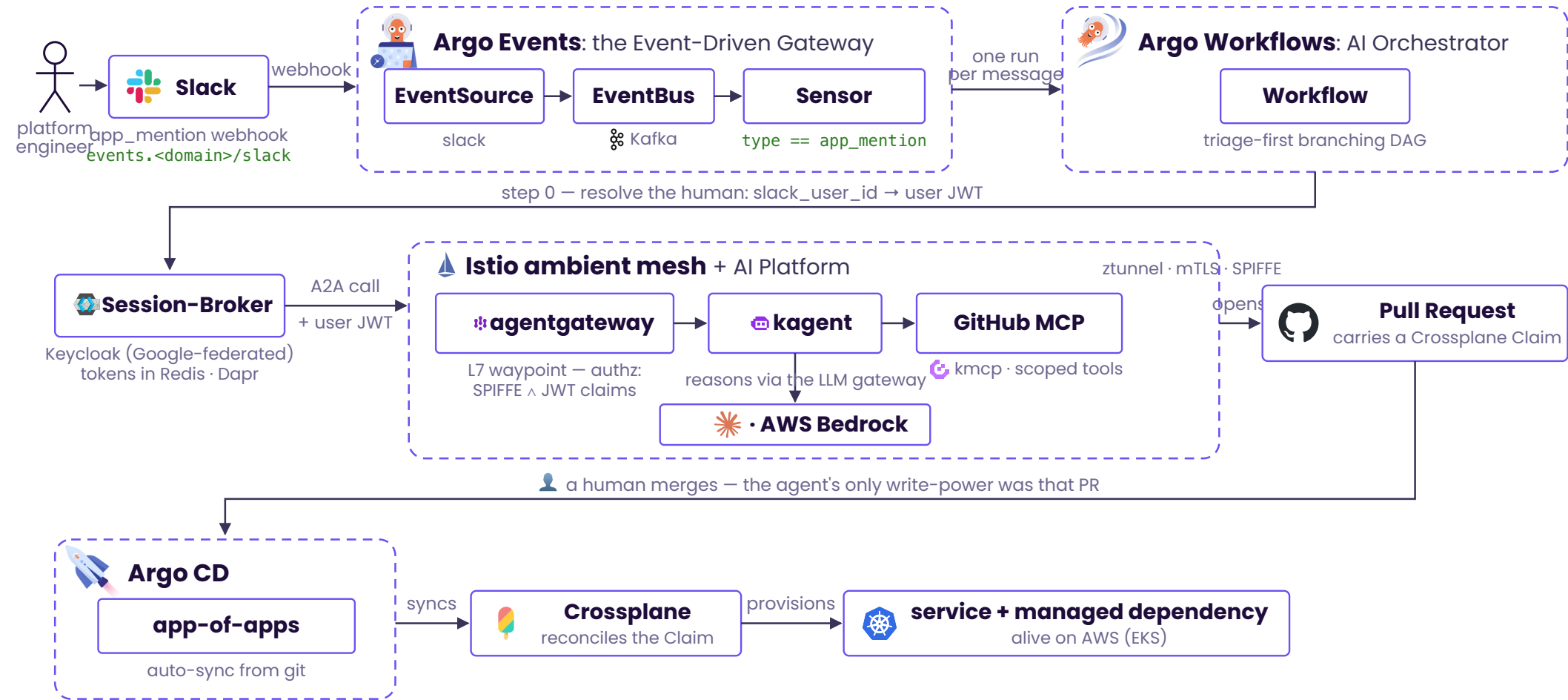


La frontera de seguridad es el **service mesh** — **nunca el modelo.**

La **IA propone**; un pipeline gobernado y auditable **dispone**.

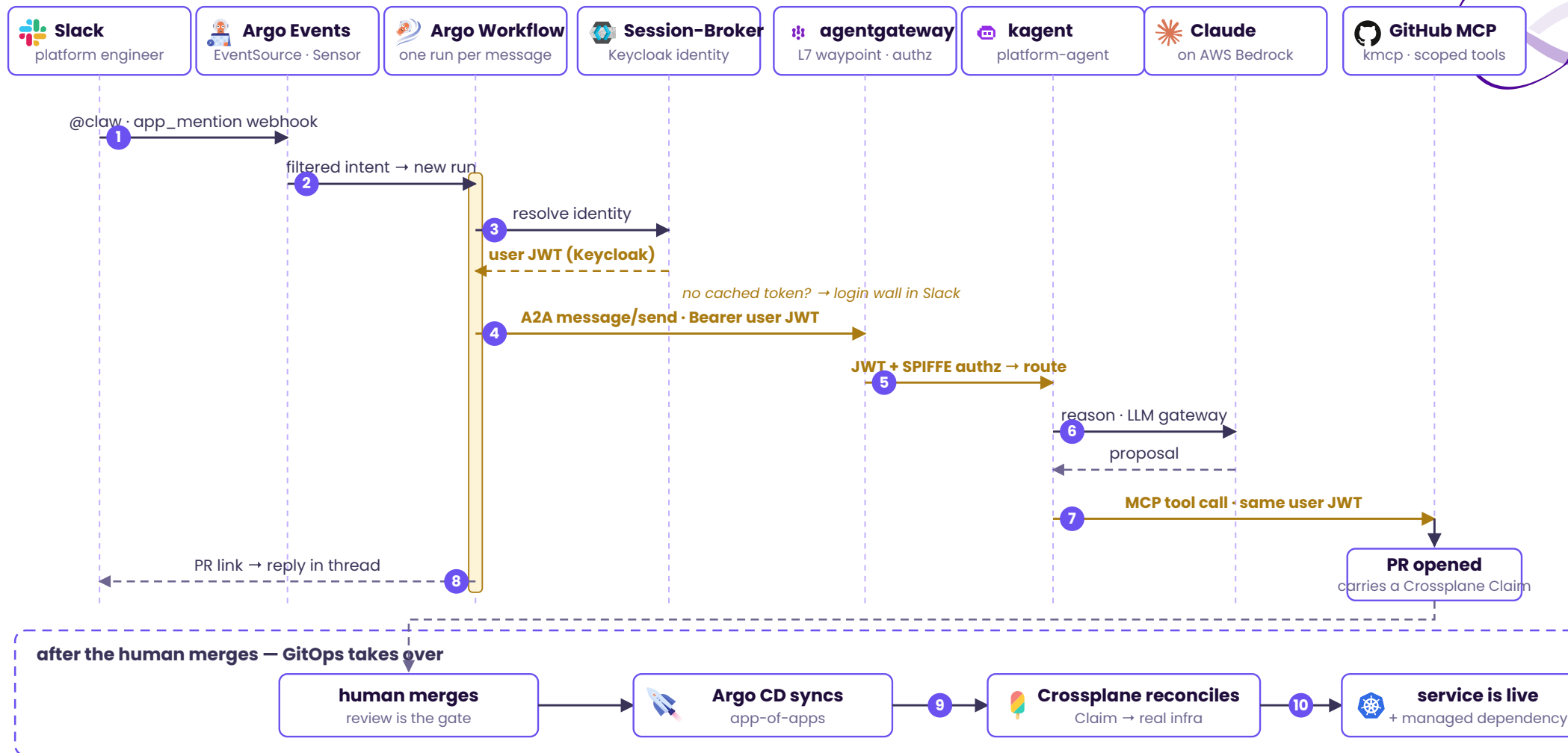
Ahora, seremos responsables. La era de «fue culpa del agente» ha terminado.

EnterpriseClaw: la arquitectura en un vistazo



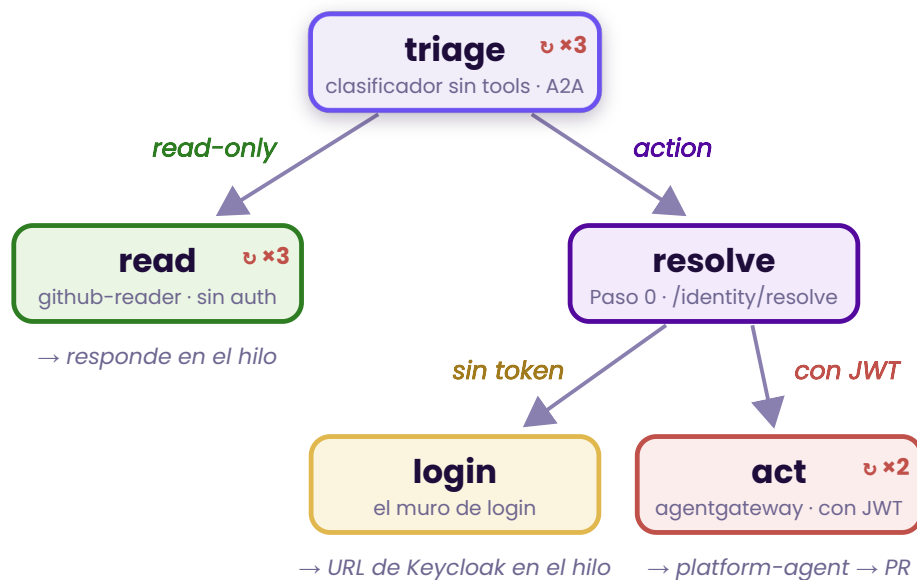
AI proposes — a governed, auditable GitOps pipeline disposes.

Una orden, de principio a fin



Principio 1 — Aislamiento: un mensaje, un run efímero

Un mensaje → **un Workflow efímero**: nace, ejecuta y muere. ¿Quién decide su camino?



trriage

rama

identidad

reintentos

```
# triage & read – ¿turno de agente inestable?  
# reenviar: las lecturas son idempotentes  
retryStrategy:  
  limit: "3"  
  retryPolicy: Always  
  backoff: { duration: 2s, factor: "2" }  
  
# act – un reintento solo re-ejecuta un  
# turno FALLIDO, que no mutó nada  
retryStrategy:  
  limit: "2"
```

Reintentos seguros por diseño — las lecturas son idempotentes; act solo re-ejecuta un turno fallido que no mutó nada.

Principio 2 — Observabilidad (y auditabilidad)



EL RASTRO ES NATIVO — SIN TOOLING ADICIONAL

RASTRO 01

El Workflow archivado

Un run por mensaje: quién lo pidió, cuándo, qué pasos corrieron y con qué resultado.

RASTRO 02

Los artefactos del run

Prompt, respuesta y manifiestos generados, guardados como artefactos en S3.

RASTRO 03

El PR + la historia de git

El cambio mismo es un diff con autor, revisión y merge — la evidencia que ya auditas.

- El auditor lee **git y la UI de Argo** — no logs de chat.
- Cada acción tiene un humano, un run y un diff con nombre.

Se acabó la era de «fue culpa del agente».



Principio 3 — Mínimos privilegios: dos rieles de identidad

Dos rieles de identidad, impuestos por **dos capas cooperantes del mismo mesh** — toda llamada A2A / MCP / LLM debe pasar por ambos:

RIEL 1 · WORKLOAD — L4 · ISTIO AMBIENT

¿QUÉ está llamando?

ztunnel da a cada pod una identidad **SPIFFE** y envuelve cada salto en **mTLS**. Agente, herramientas, broker: identidades distintas.

RIEL 2 · HUMANO — L7 · AGENTGATEWAY

¿QUIÉN está pidiendo?

El **waypoint** al que ztunnel entrega el tráfico: valida el **JWT de Keycloak** que trae el Workflow y autoriza por sus **claims** — quién pide, y qué puede alcanzar.



MIEDO 01 — NEUTRALIZADO

Prompt injection

Un prompt envenenado dobla el *razonamiento*, nunca la *identidad*. La allow-list de herramientas es **config del mesh** — fuera del alcance del modelo.



MIEDO 02 — NEUTRALIZADO

Suplantación de usuario

El JWT viene de un **login OAuth real en Keycloak**, ligado al usuario de Slack por el Session-Broker. Token forjado o ausente → **401 en el waypoint**.



MIEDO 03 — NEUTRALIZADO

Fuga de secretos

El agente no guarda **ninguna credencial**: el rol de Bedrock vive solo en agentgateway, los tokens del usuario van cifrados en Redis, mTLS cubre cada salto.



● EN VIVO

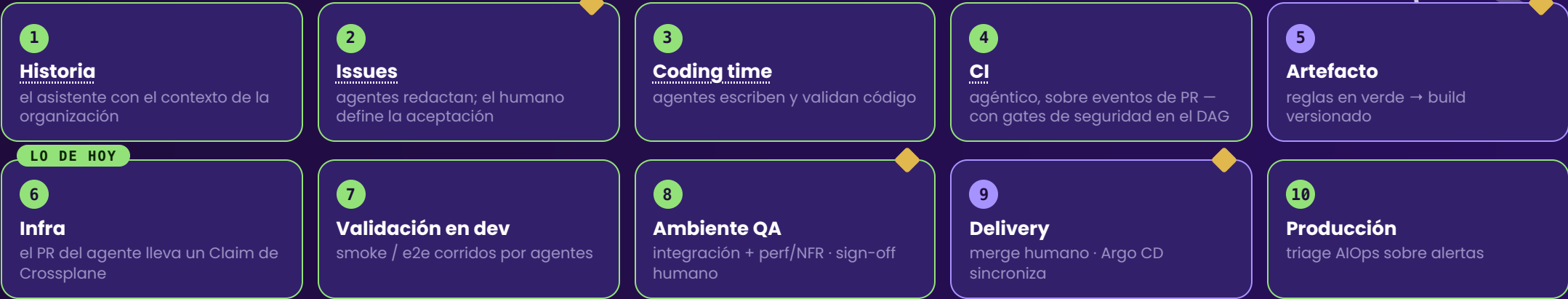
**Slack → agente → PR → merge
→ Argo CD → servicio vivo**

Guiado por las UIs: el hilo de Slack · kagent · agentgateway · Argo CD



La evolución completa: el mismo patrón por todo el SDLC

VISIÓN – HACIA DÓNDE EVOLUCIONA EL PATRÓN · HOY ESTÁ VIVO EL PASO RESALTADO



☐ **Agente** — evento → workflow efímero gobernado ☐ **Pipeline determinista** — CI / GitOps, sin agente ☒ **Gate humano** — se queda en ese paso



Slack es solo el primer EventSource: el mismo patrón gobernado — evento → intención filtrada → workflow efímero → agente con herramientas acotadas → dos rieles de identidad → rastro auditable — monta un trigger distinto en cada etapa: chat (1-2) · webhooks de PR (3-5) · eventos de registry y ambientes (6-9) · alertas (10).

Los relojes se comprimen; los rombos no se mueven.
La automatización acelera cada etapa — los gates de responsabilidad humana se quedan donde están.



PRINCIPIO (YA LO APLICAS)

EN ORQUESTACIÓN DE AGENTES

Aislamiento

Un run efímero por mensaje + segmentación del mesh.

Mínimos privilegios

Dos rieles de identidad (humano + workload) + allow-lists de herramientas impuestas en L7.

Observabilidad

Workflows archivados + artefactos + PR e historia de git como audit trail.

- EnterpriseClaw es un **patrón** sobre proyectos CNCF que probablemente ya corres (Argo, Istio, Crossplane...).
- Tu estrategia de Platform Engineering no se reemplaza — **evoluciona**: los agentes son la siguiente clase de workload de tu plataforma, y su nueva interfaz.



Gracias Conectemos

Información de contacto

LinkedIn: /in/jdarp

Juan David Argüello

InnerSource Program Lead - Credicorp

Recursos

Repos: EnterpriseClaw · Session-Broker

#DODLima #DevOpsDaysLima